

2026년 1회 정보처리기사 실기 복원 문제 (해설 추가본)

출처: <https://chobopark.tistory.com/561> (Life-Journey)

1번

다음은 C언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

double arr1(int p[], int len) {
    double av = 0;
    int i;
    for (i = 0; i < len; i++) {
        av += (double) p[i];
    }
    return av / len;
}

double arr2(int * p, int len) {
    double av = 0;
    int i;
    for (i = 0; i < len; i++) {
        av += (double)( *(p + i));
    }
    return av / len;
}

int main() {
    int arr[10] = {80, 20, 50, 55, 45, 95, 55, 10, 40, 80};
    int len = 10;
    printf("%.2f", arr1(arr, len) + arr2(arr, len));
    return 0;
}
```

정답: 106.00

해설:
두 함수 arr1과 arr2는 **사실상 똑같이 평균을 구하는** 함수입니다. 표기만 다를 뿐입니다.

- arr1은 `p[i]` 로, arr2는 `*(p + i)` 로 배열에 접근하는데, 이 둘은 **완전히 같은 표현**입니다. 둘 다 배열의 i번째 값입니다.
- 두 함수 모두 모든 원소를 더한 뒤 개수로 나눠 **평균**을 반환합니다.

먼저 배열의 합을 구합니다:
 $80+20+50+55+45+95+55+10+40+80 = 530$

평균 = $530 / 10 = 53.0$

두 함수가 같은 평균을 반환하므로:
 $arr1 + arr2 = 53.0 + 53.0 = 106.0$

`%.2f` 는 소수점 둘째 자리까지 출력 → **"106.00"**

핵심: `p[i]` 와 `*(p+i)` 는 동일한 배열 접근 방식입니다. 결국 같은 평균(53)이 두 번 더해져 106이 됩니다.

2번

다음 설명에 해당하는 디자인 패턴의 유형을 괄호() 안에 쓰시오.

- (◡) 패턴은 기능의 클래스 계층과 구현의 클래스 계층을 연결
- (◡) 패턴은 한 객체의 상태가 바뀌면 그 객체에 의존

정답: ㄱ. Bridge / ㄴ. Observer

해설:

두 가지 디자인 패턴을 구분하는 문제입니다.

- **Bridge(브리지) 패턴: 기능(추상)과 구현을 분리**하여 각각 독립적으로 확장할 수 있게 하는 **구조 패턴**입니다. '다리(Bridge)'로 두 계층을 연결한다는 의미입니다. 예를 들어 '도형(기능)'과 '색칠 방법(구현)'을 따로 관리하면, 새 도형이나 새 색칠 방식을 자유롭게 추가할 수 있습니다.
- **Observer(옵저버) 패턴: 한 객체(주제, Subject)의 상태가 바뀌면, 그것을 구독(의존)하는 다른 객체들(관찰자)에게 자동으로 알림**이 가는 **행위 패턴**입니다. 예: 유튜브 채널(주제)에 새 영상이 올라오면 구독자(관찰자)에게 알림이 가는 것.

키워드: "기능과 구현 연결 = Bridge", "상태 변화 → 의존 객체에 통보 = Observer".

3번

데이터베이스(DB) 설계 절차를 순서대로 나타낸 것이다. 각 빈칸에 들어갈 알맞은 용어를 쓰시오. (※ 원문에 이미지 포함)

정답: ㄱ. 요구사항 분석 / ㄴ. 개념적 설계 / ㄷ. 논리적 설계 / ㄹ. 물리적 설계 / ㄴ. 구현

해설:

데이터베이스 설계는 5단계를 거칩니다. 순서가 매우 자주 출제되니 꼭 외워두세요.

1. **요구사항 분석:** 사용자가 무엇을 원하는지 파악. (어떤 데이터가 필요한가?)
2. **개념적 설계:** E-R 다이어그램(개체-관계 모델)으로 데이터의 큰 그림을 그림. (현실 세계를 추상화)
3. **논리적 설계:** 개념 모델을 실제 DBMS에 맞는 **테이블 구조(릴레이션)**로 변환. 정규화 수행.
4. **물리적 설계:** 실제 저장장치에 어떻게 저장할지 결정. 인덱스, 접근 방법 등.
5. **구현:** SQL로 실제 데이터베이스를 만들고 데이터를 넣음.

암기 팁: "요구사항 → 개념 → 논리 → 물리 → 구현". 추상적인 단계에서 점점 구체적·물리적인 단계로 내려간다고 생각하면 순서가 기억됩니다.

4번

다음은 비기능적 요구사항에 대한 설명이다. 각 항목이 의미하는 요구사항 유형을 보기에서 골라 쓰시오.

1. 시스템 운영 중 로그 관리 및 모니터링 기능을 제공해야 한다.
2. 시스템 운영 시 최소 메모리 용량을 확보해야 하며, 자원 사용량은 제한 범위 내에 있어야 한다.
3. 사용자 요청에 대한 응답 시간은 최대 1분을 초과하지 않아야 한다.

보기: 신뢰성, 가용성, 운영, 유지보수성, 자원, 성능, 이식성, 보안, 품질

정답: 1. 운영 / 2. 자원 / 3. 성능

해설:

요구사항은 크게 두 종류입니다.

- **기능적 요구사항:** 시스템이 "무엇을 해야 하는가" (예: 로그인 기능, 결제 기능).
- **비기능적 요구사항:** 시스템이 "어떤 품질·제약을 가져야 하는가" (예: 빨라야 한다, 안전해야 한다).

이 문제는 비기능적 요구사항의 세부 유형을 구분합니다.

1. **운영 요구사항:** 시스템 운영·관리에 관한 것. 로그 관리, 모니터링 등.
2. **자원 요구사항:** 메모리·CPU 같은 **자원 사용량 제한**에 관한 것.
3. **성능 요구사항:** **응답 시간, 처리 속도** 등 성능에 관한 것. "응답 시간 1분 이내" = 성능.

키워드 매칭: "로그/모니터링=운영", "메모리/자원량=자원", "응답시간/속도=성능".

5번

다음 용어의 영문 약자를 쓰시오.

- 정보보호 관리체계

정답: ISMS

해설:

ISMS(Information Security Management System, 정보보호 관리체계)는 조직이 정보 자산을 안전하게 보호하기 위해 **체계적으로 관리하는 제도·시스템**입니다.

- 단순히 보안 장비를 설치하는 것이 아니라, 보안 정책 수립 → 위험 분석 → 대책 적용 → 점검·개선의 **전체 관리 과정**을 인증하는 제도입니다.
- 우리나라에서는 'ISMS-P'(개인정보보호까지 포함)라는 인증 제도로 운영됩니다.

비슷한 약자와 헷갈리지 마세요: ISMS(정보보호 관리체계), IDS(침입탐지시스템), IPS(침입방지시스템). "정보보호 관리체계 = ISMS"로 외우세요.

6번

HDLC는 비트 중심의 데이터 링크 제어 프로토콜로, 프레임 단위로 데이터를 전송하며 흐름 제어 및 오류 복구 기능을 제공한다. 다음 설명을 읽고 알맞은 용어를 쓰시오.

1. HDLC 프레임의 구성 단위로, 실제 사용자 데이터를 전송하는 프레임
2. 데이터 링크의 흐름을 관리하고 오류 제어 및 통신 상태를 감시하는 프레임
3. 순서 번호 없이 링크 설정, 해제, 모드 설정 등 제어 기능을 수행하는 프레임
4. 두 국(Station)이 동등한 위치에서 서로 명령과 응답을 주고받는 모드
5. 종국(Secondary)이 주국(Primary)의 허가 없이도 자발적으로 응답을 전송할 수 있는 모드

정답: 1. 정보 / 2. 감독 / 3. 비번호 / 4. 비동기 균형 모드 / 5. 비동기 응답 모드

해설:
HDLC 프로토콜의 프레임 종류(3가지)와 동작 모드(3가지)를 구분하는 문제입니다.

프레임 3종류:

1. **정보(Information) 프레임:** 실제 **사용자 데이터를 전송**하는 프레임. 순서 번호가 있음.
2. **감독(Supervisory) 프레임:** 데이터는 없고 **흐름 제어·오류 제어**를 담당. (응답 ACK/NAK 등)
3. **비번호(Unnumbered) 프레임:** 순서 번호 없이 **링크 설정·해제·모드 설정** 등 제어를 수행.

동작 모드:

4. **비동기 균형 모드(ABM):** 두 국이 **동등(균형)**하게 서로 명령·응답 가능. ('균형=동등')
5. **비동기 응답 모드(ARM):** 종국이 주국 **허가 없이 자발적으로** 응답 전송 가능.

암기 팁: 정보(데이터), 감독(제어/감시), 비번호(설정/해제). 균형 모드=양쪽 동등, 응답 모드=종국이 자발적 응답.

7번

다음은 Java 코드에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```
class A {
    String f(Object x) {
        return "1";
    }
    String g() {
        return f("a");
    }
}

class B extends A {
    String f(Object x) {
        return "2";
    }
    String f(String x) {
        return "3";
    }
}

public class Main {
    public static void main(String[] args) {
        A a = new B();
        System.out.println(a.g());
    }
}
```

정답: 2

해설:

오버라이딩(재정의)과 오버로딩(다중정의)이 함께 나오는 까다로운 문제입니다.

- a는 선언 타입 A, 실제 객체 B입니다.
- a.g() 호출 → g()는 A에만 있으므로 A의 g() 실행. g()는 f("a")를 호출합니다.
- 여기서 "a"는 **String** 타입입니다. 하지만 A의 g() 안에서 호출하는 f는 **컴파일 시점에 A 기준**으로 결정됩니다. A에는 f(Object x) 하나뿐이므로, 호출 대상은 **f(Object)**로 결정됩니다.
- 그런데 실행 시점에는 실제 객체가 B이므로, **f(Object)가 오버라이딩된 B의 f(Object)**가 실행됩니다 → "2" 반환.

함정 정리:

- B에는 f(String x) ("3")도 있지만, **A 기준에서 컴파일될 때 f(Object)로 결정**되어 f(String)은 선택되지 않습니다.
- 오버로딩(어떤 f를 부를지)은 컴파일 시점·선언 타입 기준, 오버라이딩(부모/자식 중 누구의 f)은 실행 시점·실제 객체 기준입니다.

따라서 f(Object)가 선택되고 → B의 f(Object) 실행 → "2".

8번

아래 파이썬 코드가 있다. 입력값으로 HumanDev를 주었을 때 출력되는 결과를 쓰시오.

```
i = input()
x = []

for word in i.split():
    x.append(word)

y = ''.join(x)
z = ''.join(c for c in y[::-1] if c not in 'ong')

print(z)
```

정답: veDamuH

해설:

문자열 뒤집기와 특정 문자 제거를 다루는 문제입니다.

입력: "HumanDev"

단계별로 따라갑니다:

- i.split() → 공백 기준으로 나눔. 공백이 없으니 ["HumanDev"].
- ''.join(x) → 다시 합침 → "HumanDev".
- y[::-1] → 문자열을 **뒤집음** → "veDnamuH".
- if c not in 'ong' → 'o', 'n', 'g' 글자는 **제외**(소문자만 해당)하고 나머지만 모음.

뒤집힌 "veDnamuH"를 한 글자씩 보며 'o','n','g'를 빼면:
- v(유지) e(유지) D(유지) **n(제거)** a(유지) m(유지) u(유지) H(유지)
- → "veDamuH"

출력: "veDamuH"

핵심: [::-1]은 뒤집기, not in 'ong'은 그 세 글자를 제외. 대문자 N은 'ong'에 없으므로(소문자만) 제거 대상이 아니라는 점도 주의하세요. (여기선 n이 소문자라 제거됨)

9번

아래 조건을 참고하여 각 SQL 구문을 실행했을 때 반환되는 행(Row)의 수를 쓰시오. (단, DEPT 칼럼은 학과명이다.)

테이블 조건: STUDENT 테이블 — 컴퓨터과 50명, 인터넷과 100명, 사무자동화과 50명

```
1. SELECT DEPT FROM STUDENT;
2. SELECT DISTINCT DEPT FROM STUDENT;
3. SELECT COUNT(DISTINCT DEPT) FROM STUDENT WHERE DEPT = '컴퓨터과';
```

정답: 1. 200 / 2. 3 / 3. 1

해설:

DISTINCT (중복 제거)와 COUNT (개수)의 동작을 묻는 문제입니다. 전체 학생은 50+100+50 = 200명입니다.

1. SELECT DEPT FROM STUDENT

- 중복 제거 없이 모든 행의 DEPT를 그대로 출력 → 200행 (학생 한 명당 한 행).

2. SELECT DISTINCT DEPT FROM STUDENT

- DISTINCT 는 중복을 제거합니다. 학과는 컴퓨터과/인터넷과/사무자동화과 3종류 → 3행.

3. SELECT COUNT(DISTINCT DEPT) FROM STUDENT WHERE DEPT = '컴퓨터과'

- WHERE로 '컴퓨터과'만 골라내면 모두 컴퓨터과(1종류뿐).

- COUNT(DISTINCT DEPT) 는 서로 다른 학과의 수를 셈 → 1종류 → 1.

핵심: DISTINCT는 중복 제거, COUNT는 개수 세기. WHERE로 컴퓨터과만 남기면 distinct한 학과는 1개뿐입니다.

10번

아래는 선수(PLAYER) 정보를 관리하는 테이블을 정의하는 SQL 문이다. 팀(TEAM) 테이블의 특정 칼럼을 참조하는 외래키 제약 조건을 추가하려 할 때, 괄호 ①~⑤에 들어갈 적절한 예약어 또는 칼럼명을 쓰시오.

조건: 외래키 제약 조건의 이름은 TEAM_TF / PLAYER 테이블의 TEAM_ID 칼럼이 외래키 / TEAM 테이블의 TEAM_ID2 칼럼을 참조

```
CREATE TABLE PLAYER (
    PLAYER_ID    CHAR(7)      NOT NULL,
    PLAYER_NAME  VARCHAR2(20) NOT NULL,
    TEAM_ID      CHAR(3)      NOT NULL,
    PRIMARY KEY  (PLAYER_ID),
    ( 1 ) TEAM_TF
    ( 2 ) KEY ( 3 )
    ( 4 ) TEAM ( 5 )
);
```

정답: 1. CONSTRAINT / 2. FOREIGN / 3. TEAM_ID / 4. REFERENCES / 5. TEAM_ID2

해설:

외래키(Foreign Key) 제약조건을 정의하는 SQL 문법을 묻는 문제입니다. 외래키는 다른 테이블의 키를 참조해 테이블 간 관계를 맺는 연결고리입니다.

외래키 정의 문법은 다음과 같습니다:

```
CONSTRAINT 제약조건이름 FOREIGN KEY (이 테이블의 컬럼)
REFERENCES 참조할테이블 (참조할 컬럼)
```

각 빈칸을 채우면:

- ① **CONSTRAINT**: 제약조건의 이름(TEAM_TF)을 지정하겠다는 키워드.
- ② **FOREIGN**: FOREIGN KEY 로 외래키임을 선언.
- ③ **TEAM_ID**: 이 PLAYER 테이블에서 외래키가 될 컬럼.
- ④ **REFERENCES**: "어느 테이블을 참조하는가"를 지정하는 키워드.
- ⑤ **TEAM_ID2**: 참조 대상인 TEAM 테이블의 컬럼.

전체를 읽으면: "CONSTRAINT TEAM_TF FOREIGN KEY (TEAM_ID) REFERENCES TEAM (TEAM_ID2)". 외래키 정의 문법의 순서(CONSTRAINT 이름 → FOREIGN KEY 내 컬럼 → REFERENCES 대상테이블 대상컬럼)를 외워주세요.

11번

두 호스트 a, b의 IP 주소와 서브넷 마스크가 주어졌을 때, 각 호스트가 속한 네트워크 주소를 CIDR 표기법으로 쓰시오.

조건: 호스트 a IP 192.168.11.20 / 호스트 b IP 192.168.12.200 / 서브넷 마스크 255.255.254.0

- a. 호스트 a(192.168.11.20)가 속한 네트워크 주소
- b. 호스트 b(192.168.12.200)가 속한 네트워크 주소

정답: a. 192.168.10.0/23 / b. 192.168.12.0/23

해설:
서브넷 마스크로 네트워크 주소를 구하고 **CIDR 표기법(/숫자)**으로 나타내는 문제입니다.

- 먼저 서브넷 마스크 255.255.**254**.0을 분석합니다.
- 254 = 2진수 11111110 . 1이 7개이므로 세 번째 옥텟에서 7비트가 네트워크용.
 - 전체 네트워크 비트 = 8 + 8 + 7 = **23** → CIDR 표기 **/23**.
 - 세 번째 옥텟이 **2씩 묶입니다** (256 - 254 = 2). 즉 0-1, 2-3, ..., 10-11, 12-13 ...

호스트 a (192.168.11.20): 세 번째 옥텟 11.

- 11이 속한 2개 묶음은 **10~11** (10부터 2개). 네트워크 시작은 더 작은 값 → 세 번째 옥텟 **10**.
- 네트워크 주소 = **192.168.10.0/23**.

호스트 b (192.168.12.200): 세 번째 옥텟 12.

- 12가 속한 묶음은 **12~13**. 시작은 12.
- 네트워크 주소 = **192.168.12.0/23**.

핵심: 마스크에서 네트워크 비트 수(/23)를 구하고, 세 번째 옥텟의 묶음 크기(2)로 어느 네트워크에 속하는지 계산합니다. 묶음의 시작값이 네트워크 주소가 됩니다.

12번

다음은 C언어에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```
struct fns {
    int* (*fn)(int*);
} mine;

int* dummy(int *d) {
    return d + 1;
}

int main() {
    struct fns mine;
    int n[] = {16, 32};
    mine.fn = dummy;
    printf("%x", *mine.fn(n));
    return 0;
}
```

정답: 20

해설:
함수 포인터와 16진수 출력이 결합된 어려운 문제입니다. 표기가 복잡하지만 차근차근 보면 됩니다.

- `mine.fn = dummy` → 구조체 멤버 fn에 dummy 함수를 연결(함수 포인터).
- `mine.fn(n)` → `dummy(n)`을 호출하는 것과 같음. `dummy`는 `return d + 1` 이므로 **배열의 1칸 뒤 주소를** 반환합니다. 즉 `n[1]`을 가리키는 주소.
- `*mine.fn(n)` → 그 주소의 값 = **`n[1]` = 32**.

이제 `%x` 는 **16진수**로 출력합니다.

- 32를 16진수로 바꾸면 $32 = 16 \times 2 + 0 = \mathbf{0x20}$ → 출력 "20".

핵심: ① 함수 포인터로 `dummy`를 호출, ② `dummy`가 `d+1`을 반환하므로 `n[1]`을 가리킴, ③ `%x` 라서 32를 16진수 "20"으로 출력. 16진수 변환($32 = 0x20$)을 기억하세요.

13번

다음은 파이썬에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```
lst = list(range(10))
for c in lst[::-2]:
    print(c, end='A')
```

```
print()
```

정답: 9A7A5A3A1A

해설:

파이썬 **슬라이싱**과 `print`의 `end` 옵션을 다루는 문제입니다.

- `list(range(10))` → [0, 1, 2, 3, 4, 5, 6, 7, 8, 9].
- `lst[::-2]` → 슬라이싱 [시작:끝:간격] 에서 간격이 -2입니다. 음수 간격은 **뒤에서부터** 진행하고, 2는 한 칸씩 건너뛴다는 뜻입니다.
- 따라서 9부터 시작해 2씩 거꾸로: **9, 7, 5, 3, 1**.

`print(c, end='A')` → 각 값 출력 후 줄바꿈 대신 'A'를 붙입니다.

- 9 출력 후 A → "9A"

- 7 출력 후 A → "7A"

- ... 반복

결과: **"9A7A5A3A1A"**

핵심: `[::-2]` 는 "뒤에서부터 2칸씩"이라 9,7,5,3,1이 나오고, `end='A'` 는 출력 뒤에 A를 붙입니다.

14번

아래 파이썬 코드를 실행했을 때 출력되는 값을 쓰시오.

```
def f(a):
    m = [[x] for x in a]
    b = m[:]
    for i in range(len(b) - 1):
        b[i+1] += b[i]
    return sum(len(x) for x in m)

print(f([1, 2, 3, 4]))
```

정답: 10

해설:

이 문제의 핵심은 **얕은 복사(shallow copy)**입니다. `b = m[:]` 는 리스트 b를 새로 만들지만, **안에 든 리스트들은 m과 똑같은 것을 공유**합니다.

단계별로 봅시다.

- `m = [[x] for x in a]` → `m = [[1], [2], [3], [4]]` (각 원소가 작은 리스트).

- `b = m[:]` → b는 새 리스트지만 **b[0]과 m[0]은 같은 [1]을 가리킴** (얕은 복사).

- 반복문 `b[i+1] += b[i]` → 리스트끼리 += 는 **내용을 이어붙임**. 그런데 `b[i+1]`은 `m[i+1]`과 같은 객체이므로 m도 함께 바뀝니다!

반복을 추적하면:

- `b[1] += b[0]` → [2] 뒤에 [1] 붙임 → [2,1]. (`m[1]`도 [2,1]이 됨)

- `b[2] += b[1]` → [3] 뒤에 [2,1] 붙임 → [3,2,1]. (`m[2]`도)

- `b[3] += b[2]` → [4] 뒤에 [3,2,1] 붙임 → [4,3,2,1]. (`m[3]`도)

이제 `m = [[1], [2,1], [3,2,1], [4,3,2,1]]`.

각 리스트의 길이: $1 + 2 + 3 + 4 = 10$.

핵심: `m[:]` 는 걸만 복사하고 내부 리스트는 공유하므로, b를 바꾸면 m도 바뀝니다. 이 '얕은 복사' 함정이 출제 의도입니다.

15번

다음은 특정 공격 기법에 대한 설명이다. 아래 내용을 읽고 해당하는 공격 기법을 [보기]에서 골라 쓰시오.

원본 데이터 파일은 별도로 존재하며, 공격자는 해당 파일의 경로를 가리키는 특수 파일을 생성한다. 프로그램이 임시 파일을 생성하는 순간을 틈타 해당 임시 파일을 미리 준비한 특수 파일로 교체한다. 이후 프로그램이 임시 파일의 존재를 확인하면 교체된 파일을 정상으로 인식하고 동작하게 된다.

[보기] 하드링크 / 심볼릭링크 / 소프트링크 / 정적링크 / 동적링크

정답: 심볼릭링크

해설:
심볼릭 링크(Symbolic Link)는 다른 파일의 **경로를 가리키는 바로가기 파일**입니다. (윈도우의 '바로가기'와 비슷한 개념)

이 공격의 핵심은:
- 공격자가 원본 파일을 가리키는 심볼릭 링크(특수 파일)를 만들어 둡니다.
- 프로그램이 임시 파일을 만드는 **찰나의 순간**을 노려, 그 임시 파일을 공격자가 준비한 심볼릭 링크로 바꿔치기합니다.
- 프로그램은 그것이 정상 임시 파일인 줄 알고 작동하지만, 실제로는 링크가 가리키는 **원본(민감한) 파일**을 건드리게 됩니다.

이는 검사 시점과 사용 시점의 시간차를 이용한 **TOCTOU(Time-of-Check to Time-of-Use)** 공격의 일종이기도 합니다.

비교: 하드 링크는 같은 실제 데이터를 직접 가리키는 또 다른 이름이고, 심볼릭 링크는 '경로를 가리키는' 링크입니다. "경로를 가리키는 특수 파일 = 심볼릭 링크"입니다.

16번

다음은 특정 보안 공격 기법에 대한 설명이다. 해당하는 공격 기법의 명칭을 쓰시오.

공격자는 목표 대상이 자주 방문하는 합법적인 웹사이트를 사전에 파악하여 악성코드를 삽입해 두고, 피해자가 접속하는 순간 악성 프로그램이 자동 설치되도록 유도한다. 불특정 다수가 아닌 특정 조직/인물을 겨냥하며 접속자의 IP나 환경 조건을 확인하여 선택적으로 실행되도록 설계하기도 한다.

정답: 워터링 홀 (Watering Hole)

해설:
워터링 홀(Watering Hole) 공격은 **표적이 자주 가는 합법적인 웹사이트에 미리 악성코드를 심어두고** 기다리는 공격입니다.

- 이름의 유래: 사자가 먹잇감을 직접 쫓지 않고, 동물들이 물을 마시러 오는 **물웅덩이(watering hole)** 근처에서 기다렸다 사냥하는 모습에서 따왔습니다.
- 공격자는 피해 대상(특정 조직/인물)이 평소 신뢰하고 방문하는 사이트를 파악한 뒤, 그 사이트를 감염시켜 함정을 놓습니다.
- 표적이 방문하는 순간 악성코드가 자동 설치되며, IP나 환경을 확인해 **특정 대상에게만 선택적으로** 작동시키기도 합니다.

비교: 피싱은 가짜 사이트로 유인, 워터링 홀은 **진짜 신뢰하는 사이트를 감염시켜** 기다림. 키워드 "자주 가는 정상 사이트에 함정 = 워터링 홀"입니다.

17번

다음은 Java 코드에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오.

```
public class Main {
    public static void main(String[] args) {
        int x1 = 9;
        int x2 = 2;
        String x3 = "3";
        System.out.println(x1 + x2 + "2" + x3);
    }
}
```

정답: 1123

해설:
자바에서 **+** 연산은 **왼쪽부터 차례로** 계산되며, **문자열을 만나면 그 뒤부터는 모두 문자열 연결**이 된다는 점이 핵심입니다.

- x1 + x2 + "2" + x3 를 왼쪽부터 계산합니다:
- x1 + x2 = 9 + 2 = **11** (둘 다 정수라 덧셈).
 - 11 + "2" → 여기서 문자열 "2"를 만남 → 11이 문자열로 바뀌어 **"112"** (연결).
 - "112" + x3 = "112" + "3" = **"1123"** (계속 연결).

출력: "1123"

핵심: 문자열을 만나기 전까지는 숫자 덧셈(9+2=11), 문자열을 만난 후로는 모두 이어붙이기("11"+"2"+"3"). **+** 의 왼쪽→오른쪽 계산 순서를 정확히 적용하세요.

18번

다음은 SQL에 대한 문제이다. 아래 코드를 확인하여 알맞는 출력값을 작성하시오. (※ 원문에 테이블 이미지 포함)


```
SELECT COUNT(*)
FROM employee e
JOIN dept d ON e.dep_id = d.dept_id
WHERE d.budget > (
    SELECT AVG(budget) FROM dept
);
```

정답: 2

해설:
조인 + 서브쿼리(평균 비교)가 결합된 SQL 문제입니다.

- 쿼리를 단계별로 해석합니다:
- 1. JOIN ... ON e.dep_id = d.dept_id → employee와 dept를 부서 id로 연결합니다.
 - 2. 서브쿼리 SELECT AVG(budget) FROM dept → 모든 부서의 **예산 평균**을 구합니다.
 - 3. WHERE d.budget > (평균) → **예산이 평균보다 큰 부서**에 속한 직원만 남깁니다.
 - 4. COUNT(*) → 그렇게 걸러진 행(직원)의 수를 셉니다.

풀이 순서:

- 먼저 dept 테이블의 budget 평균을 계산합니다.
- 그 평균보다 큰 budget을 가진 부서를 찾습니다.
- 그 부서들에 속한 직원 수를 셉니다.

원문 테이블 기준으로 그 직원 수가 **2명**입니다.

핵심: 서브쿼리(평균)를 먼저 구한 뒤, 그 값과 비교하는 바깥 쿼리를 처리합니다. 조인으로 두 테이블을 연결해 부서 예산과 직원을 매칭하는 것이 포인트입니다.

19번

다음은 통합 테스트에서 사용되는 더미 모듈에 대한 설명이다. 괄호 안에 들어갈 알맞은 용어를 쓰시오.

- (1)은/는 하위 모듈을 대신하여 단순한 결과값만 반환하도록 임시로 작성된 더미 모듈로, 하향식 통합 테스트 수행 시 필요하다.
- (2)은/는 상위 모듈을 대신하여 하위 모듈의 데이터 입력과 출력을 확인하기 위한 더미 모듈로, 상향식 통합 테스트 수행 시 필요하다.

정답: 1. 스텝 / 2. 드라이버

해설:
통합 테스트에서는 아직 완성되지 않은 모듈을 대신할 **가짜(더미) 모듈**이 필요합니다. 두 가지를 구분하는 것이 핵심입니다.

- **스텝(Stub): 하위 모듈**을 대신하는 가짜 모듈. 단순히 정해진 결과값만 돌려줍니다. **하향식(Top-Down)** 통합 테스트에서 사용. (위에서 아래로 진행하는데 아직 아래 모듈이 없으니 스텝으로 대체)
- **드라이버(Driver): 상위 모듈**을 대신하는 가짜 모듈. 하위 모듈에 데이터를 넣고 결과를 확인합니다. **상향식(Bottom-Up)** 통합 테스트에서 사용. (아래에서 위로 진행하는데 아직 위 모듈이 없으니 드라이버로 대체)

암기 팁:

- 하향식(위→아래) + 하위 모듈 대체 = **스텝**
- 상향식(아래→위) + 상위 모듈 대체 = **드라이버**

"스텝은 하위 대신, 드라이버는 상위 대신"으로 짚지어 외우세요.

20번

다음은 응집도의 유형에 대한 설명이다. 괄호 안에 들어갈 알맞은 용어를 쓰시오.

- (1)은/는 모듈이 다수의 관련 기능을 가질 때, 모듈 안의 구성요소들이 그 기능을 순차적으로 수행하는 경우의 응집도이다.
- (2)은/는 동일한 입력과 출력을 사용하여 서로 다른 기능을 수행하는 활동들이 모여 있는 경우의 응집도이다.
- (3)은/는 모듈 내부의 모든 기능이 단일한 목적을 위해 수행되는 경우의 응집도이다.

정답: 1. 절차 / 2. 교환 / 3. 기능

해설:
응집도(Cohesion)는 한 모듈 안 구성요소들이 얼마나 밀접하게 관련됐는지를 나타내며, **높을수록 좋은** 설계입니다. 유형별 정의를 구분합니다.

- 1. **절차적(Procedural) 응집도**: 모듈 안 요소들이 **정해진 순서대로 실행**되는 경우. (순서 관계는 있지만 데이터 흐름과는 무관)
- 2. **교환적(Communicational, 통신적) 응집도**: **같은 입력·출력 데이터를 사용하는** 서로 다른 기능들이 모인 경우.
- 3. **기능적(Functional) 응집도**: 모듈의 모든 요소가 **단 하나의 목적**을 위해 동작하는 경우, **가장 이상적인(높은)** 응집도입니다.

응집도 강한 순서(좋은→나쁨):

기능적 > 순차적 > 교환(통신)적 > 절차적 > 시간적 > 논리적 > 우연적

구분 팁:

- 기능적: 하나의 목적 (최선)
- 순차적: 앞 출력 → 뒤 입력 (데이터 흐름)
- 교환적: 같은 데이터 공유
- 절차적: 단지 순서대로 실행